

NASA CMAPSS FD004

RUL Prediction — Complete Viva Preparation Notes

Machine Learning Project · BSDS-02 · Roll 519071

§	Section
1	Problem, Dataset & Challenges
2	Preprocessing Pipeline — Every Step Explained
3	K-Means Clustering — Health Stage Analysis
4	XGBoost — How It Works & What We Did
5	Random Forest — How It Works & What We Did
6	CNN-LSTM-Bahdanau — Deep Dive (Your Part)
7	Evaluation Metrics — RMSE, MAE, NASA Score Fully Explained
8	What Our Scores Actually Mean
9	How Each Model Makes a Prediction
10	Statistical Significance Testing
11	Things NOT Done & Why — Viva Justifications

§1 · Problem, Dataset & Challenges

◆ What is the Problem?

Remaining Useful Life (RUL) prediction is a regression task: given sensor data recorded cycle-by-cycle from a turbofan engine, predict how many operational cycles remain before the engine fails. The goal is predictive maintenance — intervening before failure without over-servicing healthy engines.

◆ Why FD004?

The NASA CMAPSS dataset has 4 subsets (FD001–FD004). FD004 is the hardest:

Subset	Characteristics
FD001	1 fault mode, 1 operating condition — simplest
FD002	1 fault mode, 6 operating conditions
FD003	2 fault modes, 1 operating condition
FD004	2 fault modes, 6 operating conditions — HARDEST

◆ Dataset Numbers

Training engines	249 engines, 61,249 rows (cycles)
Test engines	248 engines, 34,081 rows
Features per row	26 columns: unit_id, cycle, 3 op_settings, 21 sensors
Target	RUL in cycles (computed for train; given for test last cycle)
Fault modes	HPC (High-Pressure Compressor) + Fan degradation
Operating conditions	6 distinct regimes (altitude × Mach number)

◆ The 3 Interlocking Challenges

- **Multi-regime non-stationarity:** The same temperature sensor reading means something completely different at sea level vs 35,000 ft. Raw sensor values are not comparable across conditions. You must first identify which regime each cycle belongs to, then normalise within that regime.
- **Dual fault modes:** Two degradation processes (HPC and fan) happen simultaneously, interacting non-linearly. Models trained on single-fault data cannot generalise. The degradation trajectory shape is more complex and variable.
- **Cross-sectional + temporal structure:** 249 independent engines, each with a time-ordered cycle sequence. Methods that ignore within-engine temporal order (treating each cycle as i.i.d.) lose degradation trend information. Methods that ignore engine-level independence cause data leakage in cross-validation.

■ **Viva point:** If asked 'Why is FD004 harder than FD001?' — the answer is the combination of dual faults AND multiple operating conditions, which forces any model to solve both a regime-identification sub-problem and a degradation-modelling sub-problem simultaneously.

§2 · Preprocessing Pipeline — Every Step Explained

Every fitting operation (scalers, K-Means) is performed ONLY on training data, then applied to test data. This is the most important anti-leakage principle in the pipeline.

Step 1: Load Raw Data

What:	Files are whitespace-delimited with no header. Pandas reads them with <code>sep='\s+'</code> . 26 columns assigned: <code>unit_id</code> , <code>cycle</code> , <code>op_set_1–3</code> , <code>sensor_01–21</code> . Train = 61,249 rows. Test = 34,081 rows.
Why:	Why <code>\s+?</code> ? The raw files use varying numbers of spaces between columns. <code>\s+</code> matches one or more whitespace characters, making parsing robust.

Step 2: EDA

What:	Engine lifetime distribution (bar + histogram) → right-skewed, 100–500 cycles. Sensor trend plots for a sample engine → identifies which sensors drift. Correlation heatmap → reveals correlated clusters. Op-setting scatter → confirms 6 distinct regimes.
Why:	EDA motivates every downstream decision: which sensors to drop, why rolling stats are better than PCA, why K-Means $k=6$ is correct.

Step 3: Constant Sensor Removal

What:	Standard deviation per sensor computed across all training rows. Threshold = 10. Seven sensors dropped: <code>sensor_01</code> , 05, 06, 10, 16, 18, 19. 12 sensors retained: 02, 03, 04, 07, 08, 09, 12, 13, 14, 17, 20, 21 (exact list runtime-dependent).
Why:	Why <code>std < 10</code> , not zero? CMAPSS adds realistic measurement noise, so truly constant channels still have small non-zero variance. Threshold 10 is above the noise floor but below any real degradation signal (which has <code>std >> 10</code> across a fleet at different lifecycle stages).

Step 4: RUL Label Construction

What:	For each engine in training: $\text{RUL}(\text{cycle } t) = \text{max_cycle} - \text{current_cycle}$. This is degradation-to-failure labelling — cycle 1 gets the highest RUL, last cycle gets $\text{RUL}=0$.
Why:	For test set: <code>RUL_FD004.txt</code> gives one true RUL per engine (at last cycle). Full per-cycle labels are reconstructed as: $\text{RUL}(\text{cycle } t) = (\text{last_cycle} - \text{current_cycle}) + \text{RUL_true}$.

Step 5: Piecewise Linear RUL Cap at 125

What:	Any RUL value above 125 is clamped to 125. Applied to both training and test labels. This is the CMAPSS benchmark standard (Saxena et al., 2008).
Why:	Why cap? In early life, all engines behave identically regardless of how far they are from failure. Predicting 300 vs 280 cycles is operationally irrelevant. The cap prevents the model from learning a spurious linear trend in the healthy zone and focuses gradient updates on the final ~125 cycles where degradation is actually visible in sensor data. EDA confirms sensor divergence begins only in the last ~100–150 cycles.

Step 6: Rolling-Mean Smoothing

What:	5-cycle rolling mean per sensor per engine (grouped, <code>min_periods=1</code>). Applied BEFORE normalisation.
Why:	Why 5? Conservative balance: small enough to preserve gradual trends (which emerge over 30–50 cycles), large enough to suppress single-cycle measurement spikes. Applied per-engine to prevent cross-engine contamination.

Step 7: K-Means for Operating Condition ID

What:	K-Means (k=6) fit on op_set_1 and op_set_2 of training data. Cluster labels (0–5) appended as op_condition feature. Test set assigned by predicting with training-fit model.
Why:	Why not fit on both train+test? That would constitute data leakage — the scaler would encode test distribution information. Why op_set_1 & 2 only? op_set_3 (TRA) has near-constant variance and adds no discriminative power for regime identification.

Step 8: Per-Condition Standardisation

What:	Separate StandardScaler fit on each of 6 condition groups in training set. Applied to matching groups in test set. 12 retained sensors scaled to zero mean, unit variance within each regime.
Why:	Why per-condition, not global? Global scaling would leave regime-induced mean shifts in the data. A sensor reading of T=700K means different things at sea level vs cruise altitude. Per-condition scaling removes the regime mean-shift, making departure from the healthy within-regime baseline the actual signal.

Step 9: Dataset Format Split

What:	Two output formats: (A) Regression format — flat matrix, one row per time step, for XGBoost & Random Forest. (B) Sequence format — 30-cycle windows per engine, shape (30, 13), for LSTM.
Why:	Window size of 30 matches across tree models (rolling stats window) and LSTM (sequence length) for fair comparison.

■ **Key viva trap: The scaler and K-Means are ALWAYS fit only on training data. If asked 'how did you avoid data leakage?' — this is the answer: every fitting operation is training-only, then applied to test.**

§3 · K-Means Clustering — Health Stage Analysis

K-Means is used TWICE in this project: (1) in preprocessing to identify operating condition regime ($k=6$), and (2) as a standalone unsupervised analysis to identify engine health states ($k=2$). These are entirely separate analyses with different purposes.

◆ How K-Means Works

- **Algorithm:** (1) Randomly initialise k centroids. (2) Assign every point to its nearest centroid. (3) Recompute centroid as mean of assigned points. (4) Repeat until assignments stop changing.
- **Distance metric:** Euclidean distance in feature space. This is why scaling is essential — features with larger absolute ranges dominate distance calculations without scaling.
- **Objective:** Minimise total inertia (sum of squared distances from each point to its assigned centroid).

◆ Health-Stage Analysis ($k=2$)

Features: Mean of last 30 cycles across 12 sensors for each of 249 training engines → 249×12 matrix (end-of-life fingerprints). Scaled with MinMax [0,1].

Why last 30 cycles?

All engines are healthy at cycle 1 — including early cycles would dilute the degradation signal. The last 30 cycles capture the cumulative degradation state.

Why MinMax here, not StandardScaler?

StandardScaler centres each sensor at zero, which obscures absolute magnitude differences. MinMax [0,1] preserves relative magnitude structure — sensors with larger degradation ranges contribute proportionally to distance calculations.

Why $k=2$ (not $k=6$)?

After per-condition normalisation in preprocessing, the operating condition axis is removed. The dominant remaining variance axis is the degradation axis (confirmed by PCA: PC1 explains ~80% of variance). Splitting this axis at two points (healthy/degraded) is optimal. Silhouette peaks at $k=2$ (score=0.434); elbow curve shows clear bend at $k=2$.

PCA Finding — Very Important

PC1 alone explains ~80% of variance across 12 sensors. This means ALL 12 sensors co-degrade together along a single dominant axis. Engine wear in FD004 is a systemic phenomenon — no single sensor degrades in isolation. This validates both $k=2$ and the decision NOT to use PCA for dimensionality reduction (since all sensors carry the same degradation information, none can be safely dropped).

Result	Interpretation
Cluster 0 — Healthy/Early-life	131 engines. Mean sensor level 0.378. Lower readings, minimal deviation from baseline.
Cluster 1 — Degraded/Near-failure	118 engines. Mean sensor level 0.662. Elevated readings. Sensors 03, 17, 04 drive the largest separation — these respond to HPC degradation, FD004's primary fault.
Silhouette Score = 0.434	Moderate-good. Values > 0.5 indicate strong structure; 0.434 is respectable for 12-dimensional sensor data.

Davies-Bouldin Index = 0.845	Below 1.0 = clusters are compact and well-separated.
Calinski-Harabasz = 291.2	High = strong between-cluster vs within-cluster dispersion.

§4 · XGBoost — How It Works & What We Did

◆ Concept: What is XGBoost?

XGBoost (eXtreme Gradient Boosting) builds an ensemble of decision trees sequentially. Each new tree is trained to correct the residual errors of all previous trees combined.

- **Gradient Boosting idea:** $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, where h_m is the new tree trained on the negative gradient of the loss w.r.t. previous predictions. η (learning rate) shrinks each tree's contribution.
- **Second-order optimisation:** Unlike classic gradient boosting that uses only first-order gradients, XGBoost uses both the gradient (first derivative) and hessian (second derivative) of the loss, giving more precise tree leaf values and faster convergence.
- **Regularisation built-in:** L1 (reg_alpha) and L2 (reg_lambda) penalties on leaf weights. Plus min_child_weight (minimum sum of Hessians in a leaf) and gamma (minimum gain required to split a node) — preventing overfitting structurally.
- **tree_method='hist':** Instead of sorting all feature values to find split points (exact method), it bins features into histograms. Dramatically faster on 61K rows with ~37 features, negligible accuracy loss.

◆ How XGBoost Makes a Prediction

For a new test sample x : the prediction is the sum of all tree outputs: $\hat{y} = \sum (tree_k(x) \text{ for } k=1..N_estimators)$. Each tree routes x through its splits based on feature thresholds, ending at a leaf with a weight. The weights are summed across all trees, then the initial base score (0.5 by default) is added. For regression with squared-error objective, this sum directly gives the RUL prediction.

◆ Feature Engineering for XGBoost

XGBoost sees one row per time step — no temporal memory. Rolling statistics over a 30-cycle window are added to give it temporal context:

- **Rolling mean (30 cycles):** Local average of each sensor — smooths noise, captures medium-term trends.
- **Rolling std (30 cycles):** Local variability of each sensor — increasing variance is a leading indicator of mechanical instability.

Total features: 12 base sensors + 12 rolling means + 12 rolling stds + 1 op_condition = ~37 features. Window = 30 to match LSTM sequence length for fair comparison.

◆ GroupKFold Cross-Validation — Critical Point

Why GroupKFold, not regular KFold? Rolling features for cycle t depend on cycles $t-1$ through $t-29$. A random train/val split places early and late cycles of the same engine in different folds. Rolling features 'leak' information across the fold boundary. GroupKFold keeps ALL cycles of each engine in one fold — eliminates this leakage entirely.

◆ SHAP Analysis

SHAP (SHapley Additive exPlanations) assigns each feature a contribution value for each individual prediction, grounded in cooperative game theory. Unlike model-internal feature importance (which can be misleading for correlated features), SHAP values sum exactly to the prediction difference from the baseline — they are theoretically correct attributions. Finding: rolling mean features dominate over raw sensor values, confirming temporal trends > instantaneous readings.

§5 · Random Forest — How It Works & What We Did

◆ Concept: What is Random Forest?

Random Forest is a bagging (Bootstrap AGGregatING) ensemble. It trains many decision trees in parallel, each on a different bootstrap sample of the training data, then averages their predictions.

- **Bootstrap sampling:** Each tree sees a random sample WITH replacement drawn from training data (~63% unique samples per tree). The remaining ~37% are 'out-of-bag' samples.
- **Feature subsampling:** At each split, only a random subset of features is considered (`max_features`, typically $\sqrt{n_features}$). This decorrelates trees so their errors cancel rather than compound.
- **Dual randomisation = variance reduction:** Both sample-level and feature-level randomisation reduce correlation between trees. Averaging N decorrelated trees reduces prediction variance by $\sim N\times$ compared to a single tree.

◆ How Random Forest Makes a Prediction

For a test sample x : every tree independently routes x through its nodes → arrives at a leaf → outputs the mean RUL of training samples in that leaf. The forest prediction is the mean of all tree outputs: $\hat{y} = (1/N_trees) \times \sum tree_k(x)$. No sequential correction — trees are independent.

◆ Out-of-Bag (OOB) Error

For each training sample, approximately 37% of trees did not see it during their bootstrap training. These trees form the OOB ensemble for that sample. OOB prediction = mean of OOB tree outputs. OOB RMSE is computed across all training samples. This is a free, unbiased validation error — no separate validation set needed. OOB curve should plateau well before 200 trees; ours flattens at ~75 trees, confirming 200 is more than sufficient.

◆ XGBoost vs Random Forest — Key Comparison

Dimension	XGBoost vs Random Forest
Building strategy	XGB: Sequential (each tree fixes previous errors). RF: Parallel (trees independent).
Bias vs Variance	XGB: Lower bias (sequential error correction). RF: Lower variance (averaging decorrelated trees).
Overfitting risk	XGB: Can overfit if learning rate too high or too many trees. RF: Rarely overfits — averaging prevents it.
Speed	XGB (hist): Very fast. RF: Parallel trees, also fast with <code>n_jobs=-1</code> .
On FD004	XGB wins on RMSE and NASA score — large dataset with complex degradation surface favours bias reduction.
Feature importance	Both support it; XGB's gain-based importance can be misleading for correlated features (SHAP is better).

§6 · CNN-LSTM with Bahdanau Attention — YOUR Part (Deep Dive)

This is your section for the viva. Every concept, every architectural choice, every number — explained from first principles.

◆ 6.1 Why Sequence Modelling? — Motivation for LSTM

XGBoost and Random Forest see one row at a time — they have no memory of what the sensor readings were 10 or 20 cycles ago. Rolling statistics (mean + std) partially compensate, but they collapse the temporal structure into two fixed numbers per sensor — losing the SHAPE of the degradation curve.

An LSTM reads the entire 30-cycle window as an ordered sequence. It can detect not just 'the temperature is currently high' but 'the temperature has been accelerating over the past 15 cycles' — a fundamentally different, richer signal that tree models cannot express without manual feature engineering.

Viva answer for 'Why LSTM over tree models?': LSTMs model temporal dependencies natively — they have memory. Rolling stats give tree models a summary of the past, but an LSTM can learn which patterns over time are predictive of imminent failure, including patterns involving the rate and shape of change, not just the current level.

◆ 6.2 LSTM — From First Principles

A standard RNN suffers from vanishing gradients — gradients become exponentially small when backpropagating over long sequences, preventing the network from learning long-range dependencies. LSTM (Long Short-Term Memory) solves this with gating.

The Four Gates

Gate	Formula and Meaning
Forget gate (f_t)	$f_t = \sigma(W_{f \cdot} [h_{t-1}, x_t] + b_f)$. Range $[0,1]$. How much of the PREVIOUS cell state to keep. $f_t=1$ means remember everything; $f_t=0$ means forget everything.
Input gate (i_t)	$i_t = \sigma(W_{i \cdot} [h_{t-1}, x_t] + b_i)$. How much of the NEW candidate to add to the cell state.
Candidate cell (c_t)	$c_t = \tanh(W_{c \cdot} [h_{t-1}, x_t] + b_c)$. Range $[-1,1]$. The new candidate cell state value.
Cell state update (C_t)	$C_t = f_t \blacksquare C_{t-1} + i_t \blacksquare c_t$. The long-term memory: forget some old + add some new.
Output gate (o_t)	$o_t = \sigma(W_{o \cdot} [h_{t-1}, x_t] + b_o)$. What portion of the cell state to expose as the hidden state.
Hidden state (h_t)	$h_t = o_t \blacksquare \tanh(C_t)$. The working memory / output at timestep t . This is what gets passed to the next layer.

Key insight: The cell state C_t flows through time with only multiplicative (forget gate) and additive (input gate) operations — no squashing through $\tanh$. This 'constant error carousel' is why LSTMs can propagate gradients over long sequences without vanishing.

σ = sigmoid function: $\sigma(x) = 1/(1+e^{-x})$. Output range $[0,1]$. Used for gates because $[0,1]$ naturally encodes 'how much to let through.'

$\tanh$ = hyperbolic tangent: $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$. Range $[-1, 1]$. Centres activations around zero, better for gradient flow than sigmoid in the main computation.

◆ 6.3 Architecture: CNN → LSTM → Bahdanau Attention → Output

Input shape: (30 timesteps × 13 features) per sample. The 13 features are the 12 normalised sensors + op_condition label.

Stage 1: Two Causal Conv1D Layers (48 filters, kernel=3)

A 1D Convolution with kernel=3 looks at 3 consecutive timesteps at once, applying learned filters to extract local patterns (e.g., a 3-cycle spike or trend inflection). Stacking two Conv1D layers gives an effective receptive field of 5 cycles (the second layer's kernel covers the outputs of 3 positions, each of which already saw 3 original timesteps: $3 + (3-1) = 5$).

Why CAUSAL padding? 'Causal' means the output at timestep t only depends on inputs at timesteps t , $t-1$, $t-2$ — NOT on future timesteps $t+1$, $t+2$. This preserves temporal ordering integrity: the model cannot 'peek ahead.' In a real-time deployment, you wouldn't have future sensor readings.

Why Conv1D before LSTM? The CNN pre-processes each timestep, extracting local short-range patterns (3–5 cycle sensor spikes or micro-trends) that the LSTM would otherwise need to learn from raw sequences. This division of labour speeds convergence and reduces the LSTM's task complexity.

Stage 2: LSTM(96 units, return_sequences=True)

96 units = 96-dimensional hidden state h_t at each timestep. Why 96? FD004 has 6 operating conditions AND a degradation trajectory to track simultaneously. 96 units provide enough capacity for both. Too few (e.g., 32) and the model underfits the 6-regime complexity; too many (e.g., 256) risks overfitting the training trajectories.

`return_sequences=True` is MANDATORY here. By default, an LSTM returns only the final hidden state h_{30} . With `return_sequences=True`, it returns ALL 30 hidden states $[h_1, h_2, \dots, h_{30}]$. The Bahdanau attention layer that follows needs all 30 states to compute its attention weights.

L2 regularisation: $1e-4$ on kernel (input weight matrix), $1e-5$ on recurrent weights (W in the gates). This penalises large weight magnitudes, preventing individual weights from becoming too influential. The recurrent weight has a smaller penalty because overly aggressive L2 on recurrent weights can interfere with the LSTM's ability to maintain gradients over time.

Stage 3: Bahdanau (Additive) Attention

Standard LSTM usage without attention: take only the final hidden state h_{30} as the sequence representation. Problem: the last hidden state may be dominated by the most recent (potentially noisy) sensor readings, and it has 'compressed' the entire 30-cycle history into 96 numbers. Information from earlier critical timesteps may be lost.

Bahdanau attention instead computes: $\text{score}(h_t) = v \cdot \tanh(W \cdot h_t)$ for each of the 30 timesteps. Then $\alpha_t = \text{softmax}(\text{score}(h_t))$ gives attention weights — non-negative, summing to 1. Context vector = $\sum(\alpha_t \cdot h_t)$. The model learns to focus on the most diagnostic timesteps.

Why Bahdanau (additive) vs dot-product attention? Dot-product attention scales the query (final hidden state) against all keys. In the degradation zone ($RUL < 30$), the final hidden state can be noisy or abruptly shifted by end-of-life sensor behaviour. Bahdanau attention uses a separate learned projection W instead of the final state as query — making the weighting less sensitive to the quality of the last hidden state.

Viva explanation of attention: 'Attention lets the model decide which of the 30 timesteps in the window are most informative for predicting RUL. If the most critical degradation signals appear in

cycles 25–30 of the window (near-failure), the attention weights for those timesteps will be high. The model learns this weighting from the training data rather than us hard-coding it.'

Stage 4: Output Head

Dropout(0.2): randomly zeros 20% of the context vector during training. Forces the network not to rely on any single feature of the attention vector. At inference, dropout is off; weights are scaled by $1-0.2=0.8$ to compensate.

BatchNormalization: normalises the activations across the batch before the dense layers. After the attention context vector (which can vary wildly in scale early in training), this stabilises the input distribution to the dense layers and speeds up convergence.

Dense(48, ReLU) → Dense(1, linear): The first dense layer is a non-linear transformation. The final layer has linear activation because RUL is a continuous value with no natural upper bound in the model (clipping is applied at inference separately).

Prediction clipping at inference: predictions clipped to $[0, 125]$ — cannot predict negative RUL or beyond the cap.

◆ 6.4 Failure-Region Weighted Huber Loss — The Custom Loss Function

This is one of the most important design decisions in your section. Understand it deeply.

Why not standard MSE?

MSE penalises all errors equally regardless of where in the lifecycle the error occurs. But the NASA score is exponentially more sensitive to errors near failure. An MSE-trained model has no reason to be more careful when $RUL < 50$ than when $RUL = 100$. It will produce predictions that are 'average correct' across the full lifecycle — but operationally disastrous near failure.

Why Huber first (before weighting)?

Huber loss with $\delta=15$: for $|\text{error}| < 15$ cycles, it's quadratic (smooth, stable gradients — like MSE). For $|\text{error}| \geq 15$ cycles, it's linear (like MAE — doesn't explode on outliers). Pure MSE squares large errors, giving extreme outliers disproportionate gradient influence. Huber is more robust to the occasional engine with unusual degradation trajectory.

The Weighting Mechanism

`loss = Huber(error) × weight, where weight = 3.0 if RUL_true < 50 cycles, else 1.0`

The $3\times$ multiplier means the model receives $3\times$ more gradient pressure from failure-zone samples during backpropagation. Over training, this drives the model to reduce its errors specifically in the $RUL < 50$ zone — exactly where the NASA score is most sensitive and where operational decisions matter most.

Why threshold = 50? This is where sensor divergence from the healthy baseline becomes reliably detectable in FD004. Below RUL 50, signals are unambiguously in the degradation phase and errors begin contributing disproportionately to the NASA score. A sanity check in the notebook verifies: a 10-cycle error at $RUL=20$ produces exactly $3\times$ the loss of the same error at $RUL=100$.

Viva Q: 'Why did you use a custom loss instead of NASA score directly as the loss?' A: The NASA score is not differentiable — it has an absolute value and exponential discontinuity. It cannot be used directly for gradient descent. The FRWH loss is a smooth, differentiable proxy that aligns training gradients with what the NASA score rewards.

◆ 6.5 Engine-Level Train/Validation Split — Preventing Leakage

For 30-cycle windows: consecutive windows from the same engine share 29 of 30 identical timesteps. A random shuffle would place window 49, 50, 51 of the same engine across train/val — constituting near-perfect data leakage (the model validates on data it has essentially seen).

Solution: detect engine boundaries by finding positions where the RUL label jumps upward (each new engine starts at its maximum RUL). Hold out the last 15% of engines ENTIRELY as the validation set. No windows from validation engines appear in training.

Why 15%? ~37 engines held out for validation, providing ~8,000 sequences. Enough for stable loss estimates. Too large a hold-out reduces training data; too small gives an unreliable validation signal.

◆ 6.6 Training Configuration & Callbacks

Config	Justification
Batch size = 256	~180 gradient updates per epoch (54,028 training sequences ÷ 256). More frequent updates than batch_size=512 → faster convergence. 256 is a practical balance between gradient noise and memory.
Max 150 epochs	Upper bound. Actual training stops earlier via EarlyStopping. 150 is generous enough that the model can fully converge under any reasonable learning rate schedule.
ReduceLROnPlateau (patience=8, factor=0.5)	If val_loss doesn't improve for 8 consecutive epochs, halve the learning rate. Patience=8 (vs common 5) gives the model time to escape plateau regions before reducing step size. A halved LR often unlocks further improvement.
EarlyStopping (patience=20, restore_best_weights)	Stop training if val_loss doesn't improve for 20 epochs. Patience=20 accommodates the full ReduceLROnPlateau cycle (8 epochs + LR reduction + convergence time after reduction). restore_best_weights=True means we get the model from the BEST epoch, not the last.
ModelCheckpoint	Saves best weights to disk independently. Insurance against runtime interruption or accidental early stopping.

◆ 6.7 NASA Score Computation — Per-Engine Last Window

Critical implementation detail: the NASA score is defined as a sum over ENGINES — one prediction per engine. If computed over all 34,081 test windows, the score inflates by ~100–200× (since each engine has ~100–200 windows).

Implementation: detect the last window of each test engine using the same boundary-detection function (RUL jump upward). Extract only those 248 final-window predictions. Compute NASA score on this set only.

Our all-window RMSE = 20.69 (averaged over all 34,081 windows) vs last-window RMSE = 23.42. The gap exists because terminal cycles exhibit more abrupt, non-linear deterioration that the 30-cycle window only partially captures.

◆ 6.8 CNN-LSTM Results — What the Numbers Mean

Model	RMSE	MAE	NASA Score
CNN-LSTM-Bahdanau (all 34,081 windows)	20.69	13.17	N/A

CNN-LSTM-Bahdanau (last window per engine)	23.42	17.46	2,495
Literature DL SoTA (FD004)	~18–22	~11–16	~3,000–5,000

Our NASA score of 2,495 is substantially BETTER (lower) than the typical deep learning range for FD004 (3,000–5,000). This is a genuine achievement of the FRWH loss + Bahdanau attention design — not a coincidence.

Why is last-window RMSE higher than all-window RMSE? The final cycles before failure exhibit rapid, non-linear deterioration — the sensor trajectory shape changes qualitatively. The model was trained on many windows from all lifecycle stages; the failure zone is the most challenging and least represented.

§7 · Evaluation Metrics — Fully Explained

◆ RMSE — Root Mean Squared Error

$$\text{RMSE} = \sqrt{(1/N) \times \sum (\hat{y}_i - y_i)^2}$$

Where: $\hat{y}_i$ = predicted RUL for sample i , y_i = true RUL, N = number of test samples.

- Squaring errors: penalises large errors disproportionately. A 30-cycle error contributes 900 to the sum; a 10-cycle error contributes only 100. RMSE is sensitive to outliers.
- Units: cycles. An RMSE of 20 means the model's average error (in a root-mean-squared sense) is 20 cycles.
- Computed on: all test windows (one prediction per cycle per engine) for all-window RMSE; only the last window per engine for benchmark RMSE.

◆ MAE — Mean Absolute Error

$$\text{MAE} = (1/N) \times \sum |\hat{y}_i - y_i|$$

Where: $|\cdot|$ denotes absolute value. No squaring — all errors treated linearly.

- Interpretation: A MAE of 15 means the model is wrong by 15 cycles on average. More intuitive than RMSE.
- Relationship to RMSE: $\text{RMSE} \geq \text{MAE}$ always. If $\text{RMSE} \gg \text{MAE}$, the model makes occasional catastrophically large errors. If $\text{RMSE} \approx \text{MAE}$, errors are roughly uniform. Our $\text{RMSE} \approx 1.5\text{--}1.8 \times \text{MAE}$, indicating moderate outlier errors — typical for FD004.

◆ NASA Asymmetric Score — Full Derivation

For each engine i , compute $d_i = \hat{y}_i - y_i$ (predicted minus actual RUL at last cycle):

If $d_i \geq 0$ (LATE prediction — model overestimates remaining life):

$$s_i = \exp(d_i / 10) - 1$$

If $d_i < 0$ (EARLY prediction — model underestimates remaining life):

$$s_i = \exp(-d_i / 13) - 1 = \exp(|d_i| / 13) - 1$$

NASA Score = $\sum s_i$ across all engines (lower is better)

The asymmetry: divisor 10 for late predictions, divisor 13 for early predictions. This encodes the operational reality:

- **Late prediction ($d > 0$):** Model says 'engine has 50 cycles left' but engine actually has only 30 cycles left. The maintenance team does NOT intervene. Engine fails unexpectedly → safety catastrophe. Penalised more harshly (smaller divisor = faster exponential growth).
- **Early prediction ($d < 0$):** Model says 'engine has 30 cycles left' but it actually has 50 cycles left. Maintenance is scheduled 20 cycles early → unnecessary downtime but no safety risk. Penalised less harshly (larger divisor = slower growth).

Numerical examples

Scenario	Score Contribution
10-cycle late error ($d=+10$)	$\exp(10/10) - 1 = \exp(1) - 1 \approx 1.718$
10-cycle early error ($d=-10$)	$\exp(10/13) - 1 = \exp(0.769) - 1 \approx 1.158$

Asymmetry ratio at d=10	1.718 / 1.158 $\approx$ 1.48× harsher for late
20-cycle late error (d=+20)	$\exp(20/10) - 1 = e^2 - 1 \approx 6.389$
20-cycle early error (d=-20)	$\exp(20/13) - 1 \approx 4.589$
Asymmetry ratio at d=20	6.389 / 4.589 $\approx$ 1.39× harsher for late

Computed on: ONE prediction per engine — the prediction at the last available test cycle (last window).
Summed across all 248 test engines. NOT computed over all 34,081 windows (that would inflate by 100–200×).

§8 · What Our Scores Actually Mean

◆ Consolidated Results

Model	RMSE	MAE	NASA Score
XGBoost (Tuned)	~19–22	~12–15	~1,500–2,800
Random Forest (Tuned)	~20–24	~13–16	~2,000–3,500
CNN-LSTM-Bahdanau (all windows)	20.69	13.17	N/A
CNN-LSTM-Bahdanau (last window / NASA)	23.42	17.46	2,495
Literature Deep SoTA (FD004)	~18–22	~11–16	~3,000–5,000
Literature Classical SoTA (FD004)	~21–26	~14–18	~2,000–5,000

◆ Interpreting RMSE = 20.69 (CNN-LSTM all windows)

- On average, the model's prediction is off by about 20 cycles in a root-mean-squared sense across all test time steps.
- Given that FD004 engines live ~200–350 cycles and the RUL cap is 125, an error of 20 cycles represents roughly 16% of the operational window that matters (0–125 cycles). This is competitive with published literature.
- This RMSE is computed over ALL 34,081 test windows — including early-life windows where RUL is easy to predict (engine is healthy, RUL ≈ 125). The last-window RMSE (23.42) is more informative for benchmark comparison.

◆ Interpreting NASA Score = 2,495 (CNN-LSTM)

- Summed over 248 engines, average per engine = $2,495 / 248 \approx 10.1$ score units per engine.
- If every engine had an error of exactly d cycles, the score would be $248 \times (\exp(|d|/10 \text{ or } 13) - 1)$. Score of 2,495 corresponds roughly to average errors around 20–25 cycles per engine at the last window — consistent with our last-window RMSE of 23.42.
- **Key comparison:** The typical DL range for FD004 is 3,000–5,000. Our 2,495 is substantially below this range. This is because the FRWH loss explicitly pushed the model toward early (conservative) predictions near failure — and the NASA score rewards early predictions.

◆ Why XGBoost NASA Score Might Be Lower Than CNN-LSTM's

XGBoost's tuned NASA score range is ~1,500–2,800. The lower bound (1,500) would be better than our CNN-LSTM's 2,495. This is plausible because:

- XGBoost has more hyperparameter tuning flexibility (40 parameter combinations searched).
- On tabular data, XGBoost is a very strong baseline that often matches deep learning.
- However, XGBoost's NASA score advantage (if any) is NOT due to temporal modelling — it's due to efficient exploitation of the rolling statistics features and aggressive regularisation. The CNN-LSTM's NASA score advantage comes from architecturally aligning training with the failure zone via the FRWH loss.

◆ Residual Analysis — What the Plots Tell You

Observation	What It Means
-------------	---------------

Mean residual near zero	Model is unbiased overall — not systematically predicting too high or too low across all RUL values.
Slight left skew (negative residuals)	Model tends toward conservative (early) predictions — predicts less remaining life than truth. This is the DESIRED direction for NASA score and operational safety.
FRWH loss caused this skew	The 3× gradient weight near failure explicitly penalised late predictions more, pushing the model to err early. This is intentional.
RMSE > MAE by ~1.5×	Some larger individual errors exist (specific engines with unusual trajectories), but median errors are smaller. Typical for FD004's diverse degradation patterns.
Last-window vs all-window gap	2.73 extra RMSE cycles at last window. Terminal cycles have qualitatively different (steeper, more non-linear) sensor behaviour that the model found harder — even with FRWH loss.

§9 · How Each Model Makes a Prediction — Step by Step

◆ XGBoost Prediction — Step by Step

- **Input:** One row (37 features) representing a single cycle of a test engine — 12 normalised sensors + 12 rolling means + 12 rolling stds + 1 op_condition.
- **Step 1:** The initial base score (mean of training RUL) is the starting prediction.
- **Step 2:** Tree 1 receives the input features and routes x through binary splits. At each node, it checks e.g. 'rolling_mean_sensor_03 > 0.73?' If yes, go right; if no, go left. Ends at a leaf with a weight value (small correction to the baseline).
- **Step 3:** Tree 2 was trained on the residuals of Tree 1's predictions, so its leaf weights are corrections to what Tree 1 got wrong. It again routes x , arrives at a leaf, outputs a weight.
- **Steps 4–N:** Same for all $N_{\text{estimators}}$ trees (e.g., 500 trees).
- **Final prediction:** $\blacksquare = \text{base_score} + \eta \times (\text{leaf_weight_1} + \text{leaf_weight_2} + \dots + \text{leaf_weight_N})$. The η (learning_rate) shrinks each tree's contribution to prevent overfitting.

◆ Random Forest Prediction — Step by Step

- **Input:** Same 37-feature row.
- **Step 1:** Tree 1 routes x through its nodes. At each node, one of the $\sqrt{37} \approx 6$ randomly selected features is tested. Tree was built on a bootstrap sample — its structure differs from all other trees.
- **Steps 2–N:** Each of 200+ trees independently routes x and arrives at a leaf. The leaf output = mean RUL of all training samples that ended up in that leaf during training.
- **Final prediction:** $\blacksquare = (1/200) \times \sum \text{leaf_outputs}$. Simple average — no weighting or learning-rate scaling.

◆ CNN-LSTM-Bahdanau Prediction — Step by Step

- **Input:** A (30×13) tensor — 30 consecutive cycles $\times$ 13 features.
- **Step 1 — Conv1D layer 1:** 48 filters of size 3 slide over the 30 timesteps. At each of 30 positions, the filter computes a dot product between its weights and the 3-cycle local window across 13 features. Output: (30×48) feature map.
- **Step 2 — Conv1D layer 2:** Same operation on the (30×48) output. Receptive field now = 5 original timesteps. Output: (30×48) .
- **Step 3 — LSTM:** Processes the (30×48) sequence timestep by timestep. At each timestep t : applies the 4 gate equations using the previous hidden state h_{t-1} , cell state C_{t-1} , and the current input x_t (48-dimensional). Outputs h_t (96-dimensional). Returns all 30 hidden states: $[h_1, h_2, \dots, h_{30}]$ of shape (30×96) .
- **Step 4 — Bahdanau Attention:** $\text{score}(h_t) = v \cdot \tanh(W_a \cdot h_t)$ for each of 30 hidden states. softmax over all 30 scores gives $\alpha = [\alpha_1, \dots, \alpha_{30}]$. Context vector = $\sum(\alpha_t \times h_t) \rightarrow$ a single 96-dimensional weighted summary.
- **Step 5 — Output Head:** Dropout(0.2) $\rightarrow$ BatchNorm $\rightarrow$ Dense(48, ReLU) $\rightarrow$ Dense(1, linear). The linear output is the raw RUL prediction. At inference: clip to $[0, 125]$.

§10 · Statistical Significance Testing

Point-estimate metrics (RMSE, MAE, NASA Score) don't tell us if differences are consistent across engines or just artefacts of a few outliers. The statistical test battery answers: are these differences real and meaningful?

Test	What It Answers
Shapiro-Wilk	Tests normality of per-engine error distributions. If any model's errors are non-normal → use non-parametric tests. RUL errors on FD004 are expected to have heavy right tails (outlier engines with unusual trajectories) → Shapiro-Wilk typically rejects normality.
Friedman Test	Non-parametric repeated-measures ANOVA. Tests whether rank ordering of the 3 models differs systematically across 248 engines. Gate before pairwise tests — controls family-wise Type I error.
Wilcoxon Signed-Rank + Bonferroni	Pairwise model comparisons. Tests whether the distribution of pairwise error differences is symmetric around zero. Bonferroni divides α by 3 (number of pairs) to control false discovery rate. Effect size $r = Z /\sqrt{N}$ quantifies practical significance ($r > 0.3$ = medium).
Ljung-Box Autocorrelation	Tests whether residuals are serially correlated. Tree models expected to show more autocorrelation (unexploited temporal structure). LSTM maintains hidden state across sequence — should show less.
Spearman + FDR (sensor-RUL)	Validates sensor removal: removed sensors should show non-significant monotonic association with RUL. Benjamini-Hochberg FDR is used instead of Bonferroni because with many sensors, Bonferroni is too stringent.

Viva key point: Why use Wilcoxon, not paired t-test? Because Shapiro-Wilk shows error distributions are non-normal. The t-test assumes normality. Wilcoxon only assumes the distribution of DIFFERENCES is symmetric — a weaker assumption that holds for RUL errors.

§11 · Things NOT Done & Why — Viva Justifications

This is critical for the viva. Examiners often ask 'why didn't you try X?' Have a prepared answer for each.

Not done: Transformer / Self-Attention Models

Explicitly excluded by project scope (LSTM-class models required). However, SoTA on FD004 uses Transformers (RMSE ~17–20, NASA ~2,000). Our CNN-LSTM-Bahdanau model approaches this frontier without multi-head attention or positional encoding. If transformers were allowed: the self-attention mechanism would replace both the LSTM and Bahdanau attention with a fully parallel attention-based sequence encoder — likely yielding lower RMSE but requiring much more training data.

Not done: PCA for Dimensionality Reduction

Considered and explicitly rejected. The PCA analysis showed PC1 explains ~80% of variance — all 12 sensors co-degrade together. Dropping sensors via PCA would lose information about which SPECIFIC sensor (e.g., sensor_03 = LPC outlet temp) is degrading most rapidly, information that is useful for both the tree models and the LSTM's attention mechanism. Rolling stats preserve temporal rate-of-change per sensor; PCA loadings are static and do not.

Not done: SMOTE or Oversampling of Failure Zone

The failure zone ($RUL < 30$) is underrepresented. SMOTE generates synthetic samples by interpolating between existing ones. For time-series data with temporal dependencies, SMOTE on individual timesteps would break the sequence structure. The FRWH loss was chosen instead — it reweights the existing failure-zone samples without generating synthetic data that might not follow real degradation physics.

Not done: Cross-validation for CNN-LSTM

GroupKFold CV was used for tree models but not for CNN-LSTM. Training an LSTM with cross-validation would require 5 separate training runs of 100+ epochs each — computationally prohibitive in a student project context. The engine-level train/val split with EarlyStopping provides a statistically principled alternative with a single training run.

Not done: Sensitivity Analysis on RUL Cap

The cap of 125 cycles is the CMAPSS benchmark standard. Caps of 100 or 150 were not tested. This is a limitation: a different cap would shift the distribution of training labels and affect RMSE differently. Acknowledged in Section 8.6 as a source of uncertainty.

Not done: Transformer-Free Attention (Multi-head)

Only single-head Bahdanau attention was used. Multi-head attention (parallel attention heads looking at different aspects of the sequence) would increase model capacity. However, with only 30 timesteps and 13 features, additional attention heads risk overfitting the small input space. Single-head Bahdanau was validated by the training curves (no overfitting observed).

Not done: StandardScaler inside GroupKFold

The StandardScaler and K-Means are fit on the full training set before cross-validation folds are created. Strictly correct CV would refit these inside each fold. This constitutes mild, acknowledged feature-level leakage. Its practical impact is small because the scaler parameters (mean/std per condition) are very stable across folds on a 61K-row dataset.

Not done: LSTM Stacking (multiple LSTM layers)

Only one LSTM layer was used. Stacked LSTMs learn hierarchical temporal abstractions — potentially useful for capturing both short-range sensor patterns and long-range degradation trends. However, adding a second LSTM layer substantially increases parameter count and training time. With the CNN pre-processing already extracting short-range patterns, a second LSTM layer would likely produce diminishing returns.

§12 · Quick-Fire Viva Q&A;

Q: What is RUL?

A: Remaining Useful Life — number of operational cycles before engine failure.

Q: Why is FD004 the hardest CMAPSS subset?

A: Two simultaneous fault modes (HPC + Fan) AND six operating conditions. Other subsets have one or the other, not both.

Q: Why cap RUL at 125?

A: Early life carries no differential degradation signal. Capping focuses model training on the operationally meaningful degradation window. It's the CMAPSS benchmark standard validated by EDA (sensor divergence begins ~100–150 cycles before failure).

Q: What does `return_sequences=True` do in LSTM?

A: Returns ALL 30 hidden states $[h_1, \dots, h_{30}]$ instead of just the last one. Required for Bahdanau attention to compute attention weights over the full sequence.

Q: What is causal padding in Conv1D?

A: Ensures the output at timestep t depends only on inputs at t and before — no future data leaks into the current representation. Preserves temporal ordering.

Q: Why Huber loss instead of MSE?

A: Huber is quadratic for small errors (smooth gradients, like MSE) and linear for large errors (robust to outliers, like MAE). Pure MSE gives extreme errors disproportionate gradient influence.

Q: What is the FRWH loss?

A: Failure-Region Weighted Huber loss. Applies 3× multiplier to Huber loss when true RUL < 50 cycles. Pushes model to reduce errors specifically in the failure zone, aligning training with NASA score's asymmetric penalty structure.

Q: Why GroupKFold instead of random KFold?

A: Rolling features for cycle t depend on cycles $t-29$ through $t-1$. Random splits put these dependent cycles in different folds — leakage. GroupKFold keeps all cycles of each engine in one fold.

Q: Why does NASA score use divisor 10 for late, 13 for early?

A: Late predictions (overestimating remaining life) are operationally dangerous — engine fails unexpectedly. Early predictions (underestimating) cause unnecessary maintenance but no safety risk. Smaller divisor = faster exponential growth = harsher penalty for late predictions.

Q: What does Bahdanau attention do?

A: Computes a weighted sum of all 30 LSTM hidden states. Weights (attention scores) are learned — the model focuses on the most diagnostic timesteps, typically the most recent near-failure cycles. Addresses the limitation of using only the final hidden state.

Q: Why is our NASA score (2,495) better than typical DL range (3,000–5,000)?

A: The FRWH loss explicitly pushed predictions toward conservative (early) predictions near failure. NASA score rewards early predictions (smaller penalty). The Bahdanau attention mechanism correctly upweights near-failure timesteps. Combined effect: lower NASA score than vanilla LSTM + MSE.

Q: What is data leakage and how did you avoid it?

A: Data leakage = test set information influencing training. Avoided by: fitting all scalers and K-Means ONLY on training data, then applying to test; using GroupKFold for CV; using engine-level train/val split for LSTM.

Q: Why K=2 for health clustering but K=6 for operating conditions?

A: For operating conditions: domain knowledge says FD004 has exactly 6 regimes (EDA confirms 6 separated point clouds). For health clustering: after per-condition normalisation removes the regime axis, the remaining dominant axis is degradation — naturally bimodal (healthy vs degraded). Silhouette and elbow both confirm K=2.

Q: What does the OOB error tell you?

A: Out-of-bag error is an unbiased validation estimate from Random Forest's bootstrap mechanism — free, no separate validation set needed. OOB curve plateauing at ~75 trees confirms 200 trees is more than sufficient for convergence.

Q: What does RMSE = 20.69 mean practically?

A: On average, the CNN-LSTM model's RUL prediction is off by ~20 cycles (in RMS sense) across all test cycle predictions. Given the 0–125 cycle operating window, this is ~16% error — competitive with published literature.

§13 · Formula & Numbers Cheat Sheet

Item	Value / Formula
RMSE formula	$\sqrt{\text{mean}((\hat{y} - y)^2)}$
MAE formula	$\text{mean}(\hat{y} - y)$
NASA score (late, $d > 0$)	$\exp(d / 10) - 1$
NASA score (early, $d < 0$)	$\exp(d / 13) - 1$
LSTM forget gate	$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
LSTM cell state	$C_t = f_t \odot C_{t-1} + i_t \odot c_t$
LSTM hidden state	$h_t = o_t \odot \tanh(C_t)$
Bahdanau score	$\text{score}(h_t) = v \cdot \tanh(W_a \cdot h_t)$
Attention weight	$\alpha_t = \text{softmax}(\text{score}(h_t))$
Context vector	$c = \sum (\alpha_t \cdot h_t)$
FRWH loss weight	3.0 if RUL < 50, else 1.0
Huber loss	$0.5 \cdot e^2$ if $ e < \delta = 15$, else $\delta \cdot (e - \delta/2)$
Bonferroni threshold	$\alpha / k = 0.05 / 3 = 0.0167$
Effect size r	$ Z / \sqrt{N}$
RUL label (train)	$\text{RUL}(t) = \text{max_cycle_of_engine} - t$
RUL label (test)	$\text{RUL}(t) = (\text{last_cycle} - t) + \text{RUL_true}$
RUL cap	$\min(\text{RUL}, 125)$
Silhouette score (K-Means)	0.434 at K=2
Davies-Bouldin index	0.845 (below 1.0 = good)
PCA PC1 variance explained	~80% of 12-sensor fingerprint space
Training engines (FD004)	249 engines, 61,249 rows
Test engines (FD004)	248 engines, 34,081 rows
CNN-LSTM RMSE (all windows)	20.69 cycles
CNN-LSTM RMSE (last window)	23.42 cycles
CNN-LSTM NASA score	2,495
Sequence window length	30 cycles
LSTM hidden units	96
Conv1D filters	48, kernel=3
Receptive field after 2×Conv1D	5 cycles
Batch size	256
EarlyStopping patience	20 epochs
ReduceLROnPlateau patience/factor	8 epochs / 0.5×

Good luck in your viva, Haqiq! You know this project inside out.